

Models and Practice of Neural Table Representations

Francesco Pittacolo

1 Introduction

Answering natural-language questions over relational databases is a fundamental task in data management and natural language processing. Recent advances in large language models (LLMs) have led to two main paradigms for addressing this problem, which differ in how structured data is accessed and processed.

The first paradigm, Text-to-SQL, translates natural-language questions into SQL queries that are executed by a database engine to produce the final answer. The second paradigm, Direct Table Question Answering (Table QA), bypasses query generation by providing serialized table content directly to the model, which is expected to generate the answer in a structured format.

This work compares these two approaches in a controlled experimental setting using a curated subset of the Spider benchmark. The goal is not to determine a universally superior method, but to analyze their respective strengths and limitations under consistent conditions. Particular attention is given to failure modes such as schema grounding errors, relational reasoning mistakes, context-length constraints, and output formatting issues.

Evaluation follows a data-centric methodology inspired by QATCH, using Cell Precision, Cell Recall, Tuple Cardinality, Tuple Constraint and Tuple Order. Experiments are conducted across multiple models, datasets, and prompting strategies to ensure a comprehensive comparison of both paradigms.

Overall, this study provides an empirical understanding of how large language models interact with relational data through either symbolic query generation or direct table reasoning.

2 Methodology

2.1 Overview

An evaluation framework is built around two inference pipelines for relational question answering:

- **Text-to-SQL pipeline:** SQL generation is performed from natural-language inputs and queries are executed on a SQLite backend to obtain final answers.
- **Direct Table QA pipeline:** structured table representations are provided directly to the model, which produces final answers without intermediate query execution.

2.2 Models Used

All experiments were conducted using the following models accessed through the Groq API platform:

- `openai/gpt-oss-120b`

- openai/gpt-oss-20b
- llama-3.3-70b-versatile
- llama-3.1-8b-instant
- qwen/qwen3-32b

2.3 Text-to-SQL Pipeline

2.3.1 Schema Serialization

For each question, only the oracle-relevant tables provided by the benchmark were included in the prompt context. Database schemas were serialized using a compact JSON representation containing table names and corresponding column names.

Example schema serialization:

```
{
  ...
  "schema": [
    {
      "table": "goods",
      "columns": [
        "Id",
        "Flavor",
        "Food",
        "Price"
      ]
    }
  ]
},
```

This representation was chosen because it preserves the structural organization of the database while remaining compact and easy for the language model to parse. Restricting the prompt to oracle-relevant tables also reduced prompt length and minimized distractions from unrelated schema information.

2.3.2 Prompt Design

The Text-to-SQL pipeline uses LLMs to generate SQL queries from natural-language questions. For each input instance, the model receives the serialized database schema together with the corresponding question and is instructed to produce a valid SQLite query. The prompt optionally includes a small number of in-context examples in the few-shot setting.

The model was instructed through both system-level and user-level prompts to behave as a strict SQLite query generator. The prompt explicitly constrained the model to:

- use only tables and columns appearing in the provided schema;
- generate valid SQLite syntax only;
- avoid hallucinating tables, columns, or values;
- avoid non-SQLite operators and unsupported syntax;
- avoid explanations, reasoning traces, or <think> blocks;

- return outputs exclusively in JSON format.

The output format was constrained as follows:

```
{"sql": "..."}

```

If the question could not be answered using the provided schema, the model was instructed to return:

```
{"sql": null}

```

These constraints were introduced to simplify automatic parsing and reduce formatting ambiguity during evaluation.

2.3.3 Zero-Shot and Few-Shot Prompting

Two prompting strategies were evaluated for SQL generation:

- **Zero-shot prompting:** the model receives only the serialized oracle-relevant database schema together with the target natural-language question;
- **Few-shot prompting:** the prompt additionally includes a small number of in-context examples composed of natural-language questions and their corresponding SQL queries.

Since each inference request was executed as an independent API call without conversational memory, few-shot examples were randomly sampled from the evaluation subset while ensuring that the target question itself was excluded. Each example consisted of a natural-language question paired with its corresponding SQL query.

2.3.4 SQL Generation Pipeline

The Text-to-SQL pipeline follows a multi-stage process consisting of generation, parsing, normalization, execution, and result post-processing. After prompt construction, the language model generates a response expected to contain a SQL query encoded in JSON format. The output is parsed using JSON deserialization, with lightweight extraction applied when minor formatting inconsistencies are present (e.g., surrounding text or truncated JSON structures).

In order to preserve the natural error behavior of the model, no aggressive JSON auto-repair is applied. Only minimal substring extraction is performed to recover a valid JSON object when possible. As a result, malformed outputs (e.g., truncated or structurally invalid JSON) are retained as failures and directly impact evaluation metrics.

The extracted SQL query is then normalized prior to execution to improve robustness and reduce sensitivity to superficial syntactic variations. This normalization step includes:

- removal of markdown formatting artifacts;
- standardization of function and case-related SQL expressions (e.g., LOWER/UPPER handling);
- normalization of string comparison expressions to improve case-insensitive matching.

The normalized SQL query is executed against a local SQLite database using Python’s `sqlite3` library. Queries that fail due to syntax errors, invalid schema references, or runtime exceptions are treated as incorrect predictions and contribute zero recall in evaluation.

2.3.5 Result Extraction and Normalization

The results returned by SQL execution are post-processed and normalized into a unified representation for evaluation. Since SQLite returns heterogeneous tuple-based outputs depending on query structure, a deterministic normalization step is applied to ensure comparability across different model outputs.

This step includes:

- conversion of tuples into lists;
- trimming of whitespace and type casting of values into strings;
- deterministic sorting of result tuples to ensure order-invariant comparison when required.

2.3.6 Output Representation and Logging Format

Each model inference produces a structured prediction record at the level of individual questions, containing the question identifier, the input natural language question, and a prediction dictionary.

The prediction dictionary includes:

- the generated SQL query;
- the execution output (if applicable);
- a status flag indicating success or type of failure;
- an error field capturing parsing or execution failures.

This design enables joint storage of generation and execution outcomes, allowing fine-grained analysis of model behavior beyond binary correctness. During execution, results are incrementally stored in a JSONL format, where each line corresponds to a single prediction record. Upon completion of a full run, the JSONL file is consolidated into a single JSON structure for easier downstream processing and for subsequent evaluation.

At the dataset level, all prediction records are represented as:

```
{
  "data": [
    {
      "id": ...,
      "question": "...",
      "prediction": {
        "sql": "SELECT ...",
        "answer": [
          ...
        ]
      },
      "status": "...",
      "error": null / "..."
    },
    ...
  ]
}
```

This structured format ensures reproducibility and supports systematic post-hoc analysis across different models and experimental configurations.

2.4 Direct Table Question Answering Pipeline

2.4.1 Database Conversion and Relevant Table Retrieval

To simplify table management and enable efficient retrieval of oracle-relevant tables, all SQLite databases were first converted into structured JSON files prior to inference. Each database was represented as a dictionary containing table names and their corresponding rows serialized as key-value objects.

Example database representation:

```
{
  "tables": {
    "goods": [
      {
        "Id": "20-BC-C-10",
        "Flavor": "Chocolate",
        "Food": "Cake",
        "Price": 8.95
      },
      {
        ...
      }
    ],
    "receipts": [
      {
        "ReceiptNumber": 10013,
        "Date": "17-Oct-2007",
        "CustomerId": 15
      },
      {
        ...
      }
    ]
  }
}
```

This representation preserves the original relational structure while enabling direct programmatic access to table contents without requiring SQL execution during inference.

For each sample, the benchmark-provided schema information was used to dynamically retrieve only the oracle-relevant tables associated with the current natural-language query. An automatic parser extracted the required table names from the schema metadata and selected the corresponding table contents from the JSON database representation. This design restricted the context to only relevant tables, thereby reducing prompt length, improving efficiency, and minimizing interference from unrelated database content during answer generation.

2.4.2 Table Serialization

The retrieved tables were serialized into a textual representation containing table names, column headers, and row values.

Example table serialization:

TABLE: goods

Id	Flavor	Food	Price
1	Chocolate	Cake	8.95
2	Vanilla	Cookie	5

This representation preserves both structural and cell-level information while remaining directly interpretable by the language model. Empty tables were explicitly marked using a dedicated `EMPTY` token.

To mitigate context-length limitations for large tables, an alternative compact serialization strategy was also implemented. In this representation, tables and rows are serialized using a compressed comma-separated format with reduced textual overhead. The compact serialization was automatically activated when context-length or request-size errors were encountered during inference.

2.4.3 Prompt Design

The Direct QA pipeline relies on prompt-based inference using LLMs, where the model receives serialized table contents together with a natural-language query and is required to directly generate the final answer without producing intermediate SQL queries. Optionally, a small number of in-context examples were included under the few-shot setting.

The model was instructed through both system-level and user-level prompts to behave as a strict table question answering system. The prompt explicitly constrained the model to:

- use only information appearing in the provided tables;
- avoid hallucinating values or relying on external knowledge;
- return outputs exclusively in JSON format;
- avoid explanations, reasoning traces, SQL queries, or additional text;
- return `{"answer": []}` when the correct result is empty;
- return `{"answer": null}` when the answer cannot be inferred from the provided tables.

The expected output format was:

```
{"answer": [...]}
```

These constraints simplified automatic parsing and enabled deterministic evaluation across different models and prompting configurations.

2.4.4 Zero-Shot and Few-Shot Prompting

Two prompting strategies were evaluated for direct table question answering:

- **Zero-shot prompting:** the model receives only the serialized tables together with the target natural-language query;
- **Few-shot prompting:** the prompt additionally includes a small number of in-context examples composed of natural-language queries paired with their corresponding answers. Each example also includes the corresponding schema information indicating the relevant tables and their attributes. Full table contents are not included in the few-shot examples to avoid excessively long prompts and reduce potential bias from repeated table structures.

As done for the Text-to-SQL pipeline, few-shot examples were randomly sampled from the evaluation subset while ensuring exclusion of the current sample.

2.4.5 Answer Generation and Output Parsing

After prompt construction, the language model generates a response expected to contain a JSON object encoding the predicted answer. The output is parsed using JSON deserialization, with lightweight extraction applied when minor formatting inconsistencies are present (e.g., surrounding text or partially malformed JSON structures).

To preserve the natural error behavior of the model, no aggressive JSON auto-repair mechanisms were applied. Only minimal substring extraction was performed when necessary. Consequently, malformed outputs (e.g., JSON with unbalanced or incorrect bracket structures) were retained as failures and directly affected evaluation metrics.

The parser supports both dictionary-based outputs of the form `{"answer": ...}` and direct list outputs when produced by the model.

2.4.6 Inference and Robustness Handling

During inference, automatic retry logic was implemented to handle temporary API rate limits. In addition, when context-length constraints were exceeded due to oversized serialized tables, the system dynamically switched to a compact serialization strategy to ensure compatibility with the model context window.

2.4.7 Answer Normalization

To ensure uniform evaluation across heterogeneous query types, all model outputs were normalized into a relational representation of type `List[List[str]]`. Each predicted answer is represented as a list of rows, where each row is a list of string-typed values.

The normalization procedure is defined as follows:

- dictionary-based outputs are converted by extracting only their values, ignoring keys;
- tuple and list outputs are directly transformed into lists;
- all values are cast into strings to ensure type consistency;
- leading and trailing whitespace is removed from all elements;
- scalar outputs are wrapped into singleton lists to maintain a uniform relational structure;
- `None` outputs are preserved to represent unanswerable queries.

This design ensures that all predictions, regardless of their original format, are mapped into a consistent table-like structure, enabling fair and deterministic evaluation across different models and prompting strategies.

2.4.8 Output Representation and Logging Format

Each inference produces a structured prediction record containing the sample identifier, the natural-language query, and a prediction dictionary.

The prediction dictionary includes:

- the normalized answer produced by the model;
- a status flag indicating successful parsing or type of failure;
- an error field when runtime or parsing failures occur;
- the raw model output for debugging and post-hoc analysis.

Consistently with the Text-to-SQL pipeline, during execution, prediction records are incrementally stored in JSONL format, where each line corresponds to a single processed sample. This mechanism enables interruption recovery and experiment resumption without recomputing completed predictions.

After successful completion of a full experimental run, the intermediate JSONL file is consolidated into a single JSON file for downstream evaluation and analysis.

At the dataset level, outputs are represented as:

```
{
  "data": [
    {
      "id": ...,
      "question": "...",
      "prediction": {
        "answer": [...],
        "status": "...",
        "error": null,
        "raw_output": "..."
      }
    },
    ...
  ]
}
```

This structured format supports reproducibility and facilitates systematic analysis of model predictions, parsing failures, and reasoning behavior across different models and prompting configurations.

3 Experimental Protocol

3.1 Spider Datasets

Experiments were conducted on a curated subset of the Spider benchmark composed of two datasets: **bakery_1** and **soccer_3**.

The **bakery_1** dataset represents the more structurally complex scenario. It contains four relational tables with sizes ranging from approximately 20 rows to nearly 600 rows. The schema is relatively narrow, with most tables containing between 3 and 4 columns. A total of 52 natural-language questions were evaluated on this database, covering multiple query patterns including projection, filtering, aggregation, grouping, and ordering.

The **soccer_3** dataset represents a smaller but structurally denser scenario. It contains two tables, both composed of fewer than 10 rows, but with wider schemas containing 6 and 7 columns respectively. The original Spider benchmark provides two paraphrased natural-language questions for each underlying SQL query in this dataset.

To explicitly analyze the effect of natural-language variation on model behavior, the dataset was split into two subsets:

- `soccer_3_1`
- `soccer_3_2`

Each subset contains 20 questions. Corresponding questions across the two subsets map to the same ground-truth SQL query while differing only in their natural-language formulation. For example, one subset may contain the question *"How many clubs are there?"*, while the corresponding question in the other subset is *"Count the number of clubs."*.

This design enables controlled analysis of linguistic variability independently from SQL complexity or database structure.

3.2 Data Preparation Pipeline

To construct the evaluation datasets, dedicated preprocessing scripts were implemented to transform the raw Spider files into structured evaluation-ready formats.

For the `bakery_1` dataset, a raw file containing natural-language questions and corresponding SQL queries was parsed using rule-based extraction. Questions were cleaned and grouped according to their associated SQL query, allowing the construction of multiple paraphrases per query. This process enabled the generation of both ground-truth mappings and test instances.

For the `soccer_3` dataset, the original Spider JSON annotations contained multiple databases. The data was first filtered to retain only entries corresponding to the `soccer_3` database. Questions were then grouped by identical SQL queries, yielding the previously introduced subsets `soccer_3_1` and `soccer_3_2`.

In both datasets, SQLite databases were exported into JSON format to enable direct programmatic access during inference, avoiding the need for SQL execution during preprocessing. The final processed artifacts consist of:

- ground-truth files mapping each question to its SQL query and expected execution output;
- test files containing question identifiers, natural-language questions, and oracle-relevant schemas;
- JSON-serialized database representations used by both pipelines.

This preprocessing stage ensures a consistent and reproducible evaluation setup across all experiments.

3.3 Evaluation Setup

For each question, only oracle-relevant tables provided by the Spider benchmark were included in the input context. This ensures that both pipelines operate under identical schema information and that differences in performance are not influenced by irrelevant tables.

Schemas were provided either in compact JSON format (Text-to-SQL pipeline) or through serialized table representations (Direct Table QA pipeline), depending on the approach.

All models were run with temperature set to 0 to ensure deterministic outputs and improve reproducibility across experimental runs.

3.4 Experimental Runs

Each experiment was executed independently across all combinations of model, pipeline, and prompting configuration. Since inference is stateless, each question is processed once per configuration, without repetition or sampling. No ensembling or majority voting strategies were applied.

In total, experiments were conducted across:

- 5 language models,
- 2 pipelines (Text-to-SQL and Direct Table QA),
- 2 prompting strategies (zero-shot and few-shot),
- 3 datasets (`bakery_1`, `soccer_3_1` and `soccer_3_2`).

This results in a total of 60 experimental configurations, corresponding to 20 configurations per dataset.

3.5 Evaluation Metrics

Evaluation is performed using a set of data-centric metrics inspired by the QATCH framework for structured output comparison. In particular, the Tuple Constraint metric is implemented following the original QATCH reference code, while the remaining metrics are custom implementations designed for this work but guided by the same evaluation philosophy.

All predictions and ground-truth outputs are first normalized into a unified relational representation, where each result is expressed as a list of tuples (rows) and each tuple is a list of string values. This ensures consistent and deterministic comparison across heterogeneous output formats produced by different models.

The following metrics are used:

- **Cell Precision:** proportion of predicted cell values that match ground-truth.
- **Cell Recall:** proportion of ground-truth cell values correctly recovered by the prediction.
- **Tuple Cardinality:** measures similarity in the number of tuples between prediction and ground truth.
- **Tuple Constraint:** measures the ratio between the cardinality of target tuples and predicted tuples, evaluating exact tuple-level multiplicity agreement.
- **Tuple Order:** custom implementation measuring ordering consistency between predicted and ground-truth tuples using a rank-based correlation formulation.

All metrics are computed after normalization. Invalid outputs, including parsing errors, SQL execution failures, or empty model responses, are treated as complete failures and assigned a score of zero across all metrics.

4 Results

This section presents the quantitative evaluation results across all datasets, models, prompting strategies, and pipelines. All results are reported using five data-centric metrics: Cell Precision (CP), Cell Recall (CR), Tuple Cardinality (TCa), Tuple Constraint (TCo), and Tuple Order (TO). Results are averaged over all questions in each dataset subset.

4.1 Strict vs. Filtered Table QA Evaluation

For the Direct Table QA pipeline, two evaluation modes are considered:

- **Strict**: includes all questions, even those where the model fails due to context limitations, formatting errors, or empty outputs. In such cases, metrics are computed as zero.
- **Filtered**: excludes questions that could not be processed due to context-window overflow caused by excessively long serialized table representations. These cases are rejected before inference as the input exceeds the maximum token limit.

This distinction allows separating model reasoning failures from architectural limitations related to input serialization.

4.2 bakery_1 Results

4.2.1 Text-to-SQL

Model	CP	CR	TCa	TCon	TOrd
Zero Shot					
openai/gpt-oss-120b	0.731	0.788	0.847	0.731	0.625
openai/gpt-oss-20b	0.770	0.827	0.866	0.769	0.651
llama-3.3-70b-versatile	0.665	0.705	0.764	0.648	0.585
llama-3.1-8b-instant	0.498	0.567	0.616	0.519	0.453
qwen/qwen3-32b	0.746	0.827	0.890	0.723	0.661
Few Shot					
openai/gpt-oss-120b	0.779	0.846	0.866	0.826	0.690
openai/gpt-oss-20b	0.703	0.759	0.820	0.703	0.624
llama-3.3-70b-versatile	0.622	0.647	0.705	0.608	0.531
llama-3.1-8b-instant	0.632	0.674	0.705	0.588	0.519
qwen/qwen3-32b	0.742	0.827	0.855	0.750	0.681

Table 1: Text-to-SQL results on bakery_1

4.2.2 Table QA (Strict)

Model	CP	CR	TCa	TCon	TOrd
Zero Shot					
openai/gpt-oss-120b	0.456	0.504	0.566	0.446	0.481
openai/gpt-oss-20b	0.315	0.340	0.381	0.311	0.317
llama-3.3-70b-versatile	0.547	0.566	0.621	0.499	0.610
llama-3.1-8b-instant	0.349	0.402	0.475	0.300	0.398
qwen/qwen3-32b	0.236	0.282	0.324	0.231	0.274
Few Shot					
openai/gpt-oss-120b	0.371	0.387	0.418	0.389	0.393
openai/gpt-oss-20b	0.317	0.326	0.353	0.334	0.343
llama-3.3-70b-versatile	0.511	0.624	0.661	0.504	0.548
llama-3.1-8b-instant	0.403	0.410	0.549	0.356	0.424
qwen/qwen3-32b	0.300	0.326	0.363	0.293	0.325

Table 2: Table QA (strict) results on bakery_1

4.2.3 Table QA (Filtered)

Model	Valid	Skipped	CP	CR	TCa	TCon	TOrd
Zero Shot							
openai/gpt-oss-120b	37	15	0.640	0.708	0.795	0.627	0.677
openai/gpt-oss-20b	37	15	0.442	0.478	0.536	0.438	0.446
llama-3.3-70b-versatile	52	0	0.547	0.566	0.621	0.499	0.610
llama-3.1-8b-instant	36	16	0.504	0.581	0.685	0.433	0.575
qwen/qwen3-32b	30	22	0.409	0.489	0.561	0.400	0.475
Few Shot							
openai/gpt-oss-120b	37	15	0.522	0.544	0.587	0.546	0.553
openai/gpt-oss-20b	39	13	0.422	0.435	0.471	0.445	0.458
llama-3.3-70b-versatile	52	0	0.511	0.624	0.661	0.504	0.548
llama-3.1-8b-instant	36	16	0.582	0.593	0.793	0.515	0.612
qwen/qwen3-32b	35	17	0.446	0.485	0.539	0.435	0.483

Table 3: Table QA (filtered) results on bakery_1

4.3 soccer_3_1 Results

4.3.1 Text-to-SQL

Model	CP	CR	TCa	TCon	TOrd
Zero Shot					
openai/gpt-oss-120b	0.890	0.950	0.890	0.900	0.780
openai/gpt-oss-20b	0.881	0.950	0.912	0.850	0.780
llama-3.3-70b-versatile	0.967	1.000	0.967	1.000	0.830
llama-3.1-8b-instant	0.894	1.000	0.919	0.950	0.805
qwen/qwen3-32b	0.887	0.950	0.912	0.850	0.755
Few Shot					
openai/gpt-oss-120b	0.890	0.950	0.890	0.900	0.780
openai/gpt-oss-20b	0.863	0.900	0.863	0.850	0.730
llama-3.3-70b-versatile	0.912	0.950	0.912	0.900	0.780
llama-3.1-8b-instant	0.890	0.950	0.890	0.900	0.780
qwen/qwen3-32b	0.875	0.900	0.900	0.850	0.705

Table 4: Text-to-SQL results on soccer_3_1

4.3.2 Table QA (Strict)

Model	CP	CR	TCa	TCon	TOrd
Zero Shot					
openai/gpt-oss-120b	0.887	0.950	0.887	0.900	0.938
openai/gpt-oss-20b	0.840	0.915	0.892	0.865	0.912
llama-3.3-70b-versatile	0.799	0.900	0.849	0.900	0.853
llama-3.1-8b-instant	0.602	0.887	0.704	0.675	0.785
qwen/qwen3-32b	0.887	0.950	0.887	0.900	0.945
Few Shot					
openai/gpt-oss-120b	0.838	0.900	0.838	0.850	0.825
openai/gpt-oss-20b	0.882	0.940	0.892	0.940	0.920
llama-3.3-70b-versatile	0.717	0.800	0.767	0.800	0.752
llama-3.1-8b-instant	0.764	0.875	0.818	0.825	0.828
qwen/qwen3-32b	0.863	0.950	0.887	0.850	0.917

Table 5: Table QA (strict) results on soccer_3_1

4.3.3 Table QA (Filtered)

Model	Valid	Skipped	CP	CR	TCa	TCon	TOrd
Zero Shot							
openai/gpt-oss-120b	20	0	0.887	0.950	0.887	0.900	0.938
openai/gpt-oss-20b	20	0	0.840	0.915	0.892	0.865	0.912
llama-3.3-70b-versatile	20	0	0.799	0.900	0.849	0.900	0.853
llama-3.1-8b-instant	20	0	0.602	0.887	0.704	0.675	0.785
qwen/qwen3-32b	20	0	0.887	0.950	0.887	0.900	0.945
Few Shot							
openai/gpt-oss-120b	20	0	0.838	0.900	0.838	0.850	0.825
openai/gpt-oss-20b	20	0	0.882	0.940	0.892	0.940	0.920
llama-3.3-70b-versatile	20	0	0.717	0.800	0.767	0.800	0.752
llama-3.1-8b-instant	20	0	0.764	0.875	0.818	0.825	0.828
qwen/qwen3-32b	20	0	0.863	0.950	0.887	0.850	0.917

Table 6: Table QA (filtered) results on soccer_3_1

4.4 soccer_3_2 Results

4.4.1 Text-to-SQL

Model	CP	CR	TCa	TCon	TOrd
Zero Shot					
openai/gpt-oss-120b	0.925	0.950	0.925	0.950	0.945
openai/gpt-oss-20b	0.890	0.940	0.925	0.890	0.920
llama-3.3-70b-versatile	0.747	0.775	0.734	0.758	0.800
llama-3.1-8b-instant	0.608	0.846	0.770	0.583	0.725
qwen/qwen3-32b	0.925	0.950	0.925	0.950	0.935
Few Shot					
openai/gpt-oss-120b	0.875	0.900	0.875	0.900	0.885
openai/gpt-oss-20b	0.925	0.950	0.925	0.950	0.945
llama-3.3-70b-versatile	0.687	0.750	0.787	0.750	0.695
llama-3.1-8b-instant	0.737	0.915	0.791	0.798	0.830
qwen/qwen3-32b	0.903	0.950	0.903	0.950	0.945

Table 7: Text-to-SQL results on soccer_3_2

4.4.2 Table QA (Strict)

Model	CP	CR	TCa	TCon	TOrd
Zero Shot					
openai/gpt-oss-120b	0.925	0.950	0.925	0.950	0.945
openai/gpt-oss-20b	0.890	0.940	0.925	0.890	0.920
llama-3.3-70b-versatile	0.747	0.775	0.734	0.758	0.800
llama-3.1-8b-instant	0.608	0.846	0.770	0.583	0.725
qwen/qwen3-32b	0.925	0.950	0.925	0.950	0.935
Few Shot					
openai/gpt-oss-120b	0.875	0.900	0.875	0.900	0.885
openai/gpt-oss-20b	0.925	0.950	0.925	0.950	0.945
llama-3.3-70b-versatile	0.687	0.750	0.787	0.750	0.695
llama-3.1-8b-instant	0.737	0.915	0.791	0.798	0.830
qwen/qwen3-32b	0.903	0.950	0.903	0.950	0.945

Table 8: Table QA (strict) results on soccer_3_2

4.4.3 Table QA (Filtered)

Model	Valid	Skipped	CP	CR	TCa	TCon	TOrd
Zero Shot							
openai/gpt-oss-120b	20	0	0.925	0.950	0.925	0.950	0.945
openai/gpt-oss-20b	20	0	0.890	0.940	0.925	0.890	0.920
llama-3.3-70b-versatile	20	0	0.747	0.775	0.734	0.758	0.800
llama-3.1-8b-instant	20	0	0.608	0.846	0.770	0.583	0.725
qwen/qwen3-32b	20	0	0.925	0.950	0.925	0.950	0.935
Few Shot							
openai/gpt-oss-120b	20	0	0.875	0.900	0.875	0.900	0.885
openai/gpt-oss-20b	20	0	0.925	0.950	0.925	0.950	0.945
llama-3.3-70b-versatile	20	0	0.687	0.750	0.787	0.750	0.695
llama-3.1-8b-instant	20	0	0.737	0.915	0.791	0.798	0.830
qwen/qwen3-32b	20	0	0.903	0.950	0.903	0.950	0.945

Table 9: Table QA (filtered) results on soccer_3_2

4.5 Cross-Pipeline Performance Interpretation

Across all datasets, Text-to-SQL consistently achieves better overall performance than Direct Table QA across most models and evaluation metrics.

On the more complex **bakery_1** dataset, the performance gap is substantial. Text-to-SQL significantly outperforms Direct Table QA in all models, particularly in Cell Precision and Cell Recall. This indicates that translating natural language into executable SQL queries provides stronger robustness for multi-table relational reasoning, as database execution enforces structural correctness and reduces ambiguity in intermediate reasoning steps.

On the simpler **soccer_3** datasets, Direct Table QA becomes more competitive, and in some configurations approaches or occasionally matches Text-to-SQL performance in tuple-level metrics. However, Text-to-SQL still remains the more stable and consistently higher-performing approach overall. This suggests that even in low-complexity settings, symbolic query execution maintains an advantage in terms of reliability and structured reasoning.

Overall, the results show a clear and systematic advantage for Text-to-SQL, while Direct Table

QA becomes viable primarily in low-complexity scenarios where the entire relational context fits within the model input.

5 Error Analysis

This section analyzes the main failure modes observed in both pipelines. Errors are organized by category and their impact is compared across Text-to-SQL and Direct Table QA approaches.

5.1 Text-to-SQL

Errors across datasets are categorized into schema grounding issues, join construction errors, SQL syntax failures, and ambiguity-related errors.

5.1.1 bakery_1 Dataset

Schema Structure.

The `bakery_1` dataset consists of three relational tables:

```
goods(Id, Flavor, Food, Price)
receipts(ReceiptNumber, Date, CustomerId)
items(Receipt, Ordinal, Item)
```

The dataset requires multi-hop reasoning across tables, making it sensitive to schema grounding and join construction errors.

Join Construction Errors.

Query: Find the receipt numbers where both Cake and Cookie were bought.

Generated SQL:

```
SELECT T1.ReceiptNumber
FROM items AS T1
JOIN items AS T2 ON T1.Receipt = T2.Receipt
WHERE LOWER(T1.Item) = LOWER('Cake')
AND LOWER(T2.Item) = LOWER('Cookie')
```

Error:

no such column: T1.ReceiptNumber

This error occurs because the model correctly constructs a self-join over the `items` table to identify co-occurrence of items within the same receipt, but incorrectly assumes the existence of a `ReceiptNumber` attribute in the `items` relation. This indicates a schema grounding failure rather than a join construction error, as the relational logic of the query is correct but the column-level mapping is invalid.

Schema Grounding Errors.

Query: Find all receipt numbers in which customer id 5 purchased Croissant.

Generated SQL:

```
SELECT T1.ReceiptNumber
FROM receipts AS T1
INNER JOIN goods AS T2
ON T1.CustomerId = 5 AND T2.Id = T1.Id
WHERE LOWER(T2.Food) = LOWER('Croissant')
```

Error:

no such column: T1.Id

This error occurs because the model correctly constructs a self-join over the `items` table to identify co-occurrence of items within the same receipt, but incorrectly assumes the existence of a `ReceiptNumber` attribute. The relational reasoning is correct, but the projection step reflects a schema grounding failure, as the model fails to map the natural language “receipt number” to the correct schema attribute `Receipt`.

SQL Syntax Errors.

Query: List the distinct ids of all customers who bought a lemon cake.

Generated SQL:

```
SELECT DISTINCT r.CustomerId
FROM receipts r
JOIN items i ON r.ReceiptNumber = i.Receipt
JOIN goods g ON i.Item = g.Id
WHERE LOWER(g.Flavor) = LOWER('lemon')
AND g.Food = 'cake'
```

Error:

unrecognized token: "'cake"

This error is caused by an unclosed string literal, reflecting token-level generation issues rather than incorrect reasoning about the query.

Logical Composition Errors.

Query: Find all receipts which either has the earliest date or has a good with price above 10.

Generated SQL:

```
SELECT T1.Receipt
FROM items AS T1
INNER JOIN goods AS T2 ON T1.Item = T2.Id
WHERE T2.Price > 10
UNION
SELECT Receipt
FROM items
ORDER BY Ordinal
LIMIT 1
```

Error:

1st ORDER BY term does not match any column in the result set

This indicates incorrect handling of compound queries. The model applies ordering over a UNION result without ensuring that the ORDER BY clause is compatible with the final projected schema.

5.1.2 soccer_3 Dataset**Schema Structure.**

The `soccer` dataset consists of two tables:


```
club(Club_ID, Name, Manager, Captain, Manufacturer, Sponsor)
```

```
player(Player_ID, Name, Country, Earnings,
        Events_number, Wins_count, Club_ID)
```

Column Ambiguity Errors.

Query: Show names of clubs that have players with more than 2 win counts.

Generated SQL:

```
SELECT DISTINCT Name
FROM club
JOIN player ON club.Club_ID = player.Club_ID
WHERE player.Wins_count > 2
```

Error:

ambiguous column name: Name

This occurs because both tables contain a **Name** column, and the model fails to disambiguate it using explicit table qualification (e.g., `club.Name`). This reflects a limitation in precise schema referencing even in simple relational settings.

5.1.3 Summary of Failure Modes

Across both datasets, four major categories of errors are identified:

- **Schema grounding errors:** incorrect mapping between natural language and schema attributes;
- **Join construction errors:** incorrect or incomplete relational joins;
- **SQL syntax errors:** malformed or incomplete SQL generation;
- **Logical composition errors:** incorrect handling of aggregation, ordering, and set operations.

These results suggest that Text-to-SQL failures are primarily driven by incorrect schema interpretation and relational reasoning, rather than natural language understanding alone.

5.2 Table QA

5.2.1 bakery_1 Dataset

Bakery dataset failure modes are analyzed. Unlike Text-to-SQL, errors arise primarily from context limits, output formatting, and structured reasoning inconsistencies rather than schema grounding.

Context Window / Token Limit Errors.

Query: For each type of food, tell me how many customers have ever bought it.

Error:

Error code: 413 — Request too large for model (TPM limit exceeded)

This failure is caused by exceeding the model's token limit (8607 requested vs 6000 allowed). It indicates that large serialized table inputs can lead to system-level failures before reasoning is performed.

JSON Structure Errors.

Query: For each type of food, what are the average, maximum and minimum price?

Raw Output:

```
{"answer": [{"Cake": {...}}, {"Eclair": {...}}, {"Tart": {...}}, ... ]}
```

Error:

```
json_error
```

This failure is caused by malformed nested JSON structures, where brackets are inconsistently closed and dictionary nesting becomes invalid. It reflects limitations in generating deeply structured aggregate outputs.

Parsing Errors with Code Generation.

Query: What is the item that was bought the least number of times?

Raw Output:

```
SELECT Item
FROM (
    SELECT Item, COUNT(*) AS cnt
    FROM your_table
    GROUP BY Item
) AS sub
WHERE cnt = (
    SELECT MIN(cnt)
    FROM (
        SELECT Item, COUNT(*) AS cnt
        FROM your_table
        GROUP BY Item
    ) AS sub2
);
```

Error:

```
parse_error
```

This error occurs because the model generates a SQL query instead of the required JSON-formatted answer. Since the evaluation pipeline expects structured JSON output, the response cannot be parsed even though it follows a valid SQL syntax.

Parsing Errors due to Truncation.

Query: Return the ordered list of all good ids.

Raw Output:

```
{"answer": [[20-BC-C-10], [20-BC-L-10], ... , [90"
```

Error:

```
parse_error
```

This failure is caused by output truncation, leading to incomplete JSON serialization. The model begins generating a valid structured list but fails to complete it due to token or decoding limits.

Structured Reasoning + Formatting Errors.

Query: What are the goods that have Apricot flavor and are cheaper than 5 dollars?

Raw Output:

```
{"answer": [[90-APR-PF,3.25],[50-APR,1.45],[51-APR,1.15]]}
```

Error:

json_error

This error arises from missing quotation marks around string identifiers, causing invalid JSON despite correct logical extraction of results.

Reasoning + Mixed Output Failure.

Query: What is the most expensive cake and its flavor?

Raw Output:

```
<think>
...
SELECT MAX(Price) FROM goods WHERE Food = 'Cake'
...based on the provided data.
Let me list outall the entries grouped by Food...
```

Error:

json_error

This failure occurs due to mixed reasoning traces and SQL generation within a structured output setting. The presence of intermediate reasoning breaks strict JSON formatting requirements.

5.2.2 soccer_3 Dataset

The soccer dataset exhibits failures primarily related to structured output contamination, where the model produces valid JSON fragments mixed with extraneous reasoning traces and repeated post-processing text. This leads to strict parsing failures even when the final extracted answer is correct. **Structured Output Contamination / JSON Error.**

Query: Show names of clubs in descending order of average earnings of players belonging.

Raw Output:

```
{"answer": [{"Chelsea"}, {"Arsenal"}, {"Blackburn Rovers"}, {"Everton"}, {"Fulham"}, {"Hull City"}, {"Liverpool"}]}
```

Corrected Answer:

```
{"answer": [{"Chelsea"}, {"Arsenal"}, {"Blackburn Rovers"}, {"Everton"}, {"Fulham"}, {"Hull City"}, {"Liverpool"}]}
```

Actual Answer:

```
{"answer": [{"Chelsea"}, {"Arsenal"}, {"Blackburn Rovers"}, {"Everton"}, {"Fulham"}, {"Hull City"}, {"Liverpool"}]}
```

...

Error:

json_error

This failure is caused by the model appending multiple structured answer blocks (e.g., “Corrected Answer” and “Actual Answer”) after an initially valid JSON object. Although the first JSON fragment is correct, the presence of additional JSON-like structures prevents reliable extraction of a single valid object. Due to the evaluation pipeline’s strict single-object parsing requirement, the response is rejected as a JSON decoding error.

5.2.3 Summary of Table QA Failure Modes

Across the bakery dataset, Table QA failures can be grouped into:

- **Context-window limitations:** token overflow leading to request failure;
- **Parsing errors:** invalid or incomplete JSON outputs;
- **Serialization errors:** malformed nested structures;
- **Truncation errors:** incomplete responses due to generation limits;
- **Reasoning-format conflicts:** mixing reasoning traces with structured output.

Overall, Table QA errors are dominated by output formatting and context constraints rather than schema understanding or relational reasoning.

5.3 Cross-Pipeline Error Comparison

Text-to-SQL generates executable SQL queries that are run directly on the database. This reduces the risk of hallucinated final answers, since outputs are grounded in query execution results. However, the model can still produce schema-level errors, such as incorrect or non-existent column references, which lead to execution failures.

In contrast, Table QA operates directly on serialized table content provided in the prompt. This allows the model to access all relevant data explicitly and often simplifies reasoning for small or medium-sized inputs. However, this comes at a significantly higher token cost, especially for large tables, and introduces scalability limitations due to context window constraints.

Moreover, Table QA raises additional concerns in privacy-sensitive settings, since raw tabular data must be included in the prompt. In such cases (e.g., private or institutional datasets), using a local model becomes preferable to avoid exposing sensitive information externally.

In terms of error behavior, Table QA also exhibits a higher rate of formatting and serialization failures, since outputs must strictly conform to JSON structures. These issues include invalid JSON formatting, truncation, and structured decoding errors.

Overall, Text-to-SQL is more efficient and robust in terms of scalability and data exposure, while Table QA is more direct but incurs higher computational cost and stricter output formatting constraints.

6 Code Availability

The full implementation is publicly available at:

<https://github.com/francesco-pittacolo/neural-table-representations>.

Instructions for reproducing the experiments are included in the repository documentation.